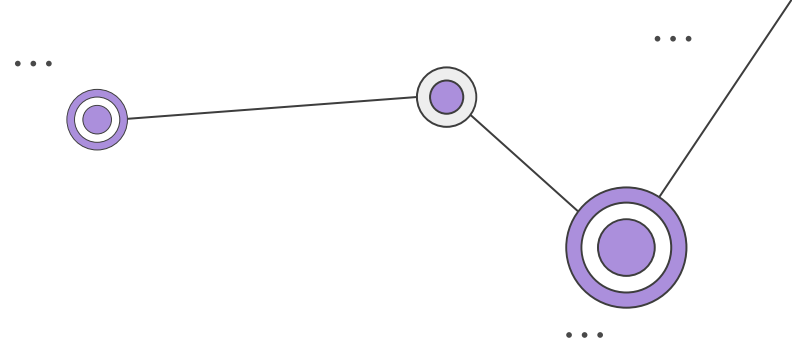
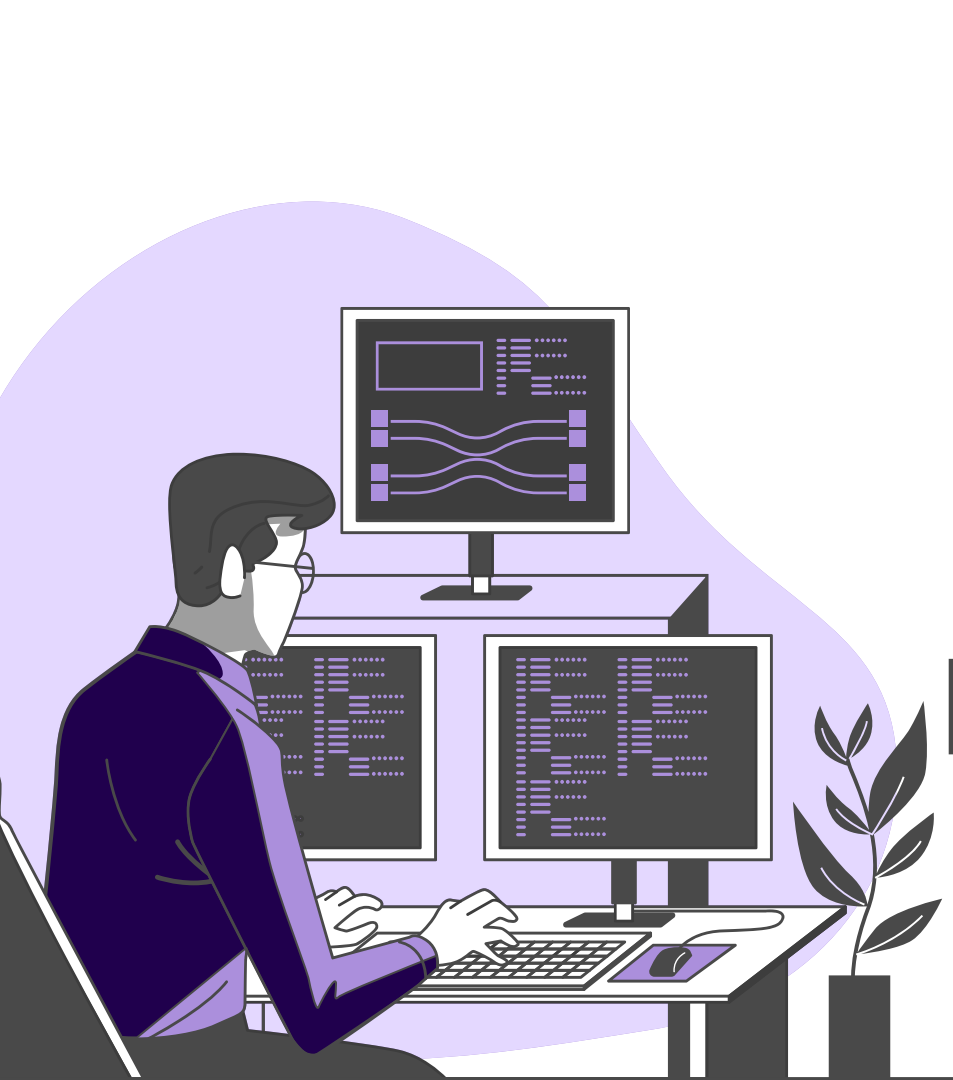
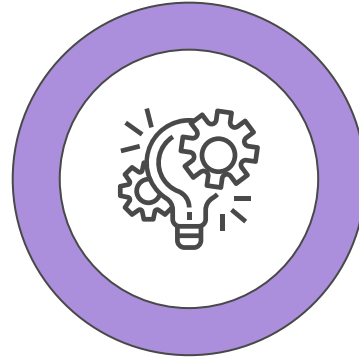




Ayudantía 12: Diagramación UML 5

Martín Eyzaguirre



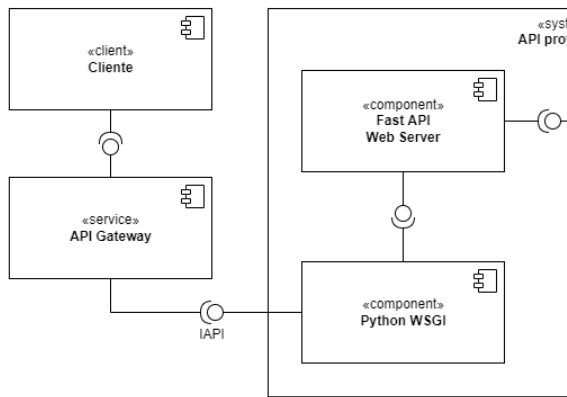


Dudas

Interrogaciones
Actividades/Debates
Programa
Dudas pequeñas
Etc
...

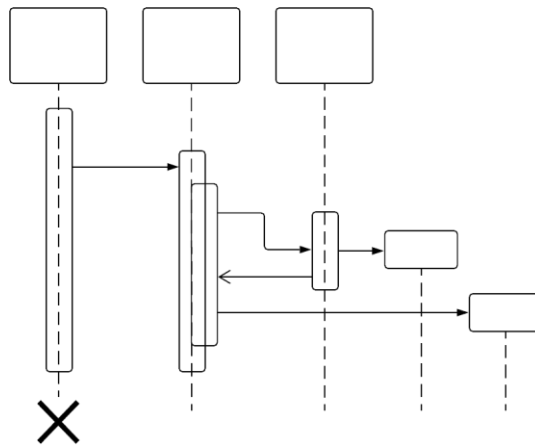
Diagramas importantes para este curso

Component Diagram (estructura)



Conectores (Cómo interactúan)
Componentes (Cajas y sus subcajas)
Interfaces (C requiere y O provee)

Sequence Diagram (comportamiento)



Actores (Columnas)
Mensajes (Señales entre columnas)

Métodos clave



Inductivo: detalle a lo general

Tengo que armar un chat en tiempo real: debe aplicar invocación implícita

Tengo que almacenar fotos: hay que introducir un tipo de storage

...



Deductivo: general al detalle

Debo cubrir 1000 usuarios por hora: un balanceador de carga

Debo tener una arquitectura orientada a servicios (SoA)

...



Referencia

He visto esto en otro lado...

Hay componentes reusables parecidos

...

Métodos clave

Performance

Cache [llave-valor]
Workers

Base de Datos

No SQL
SQL (Soportan ACID)

Escalabilidad

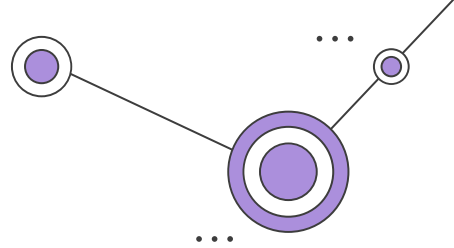
Horizontal
Vertical

Seguridad

Firewall
HTTPS
Auth

Repasos adicionales

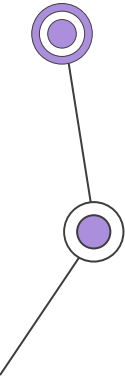
Performance



¿Qué es performance?

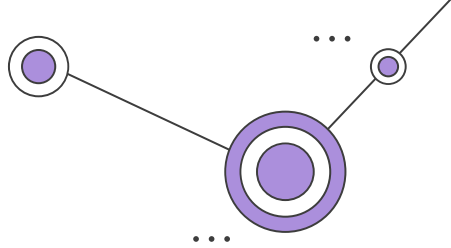
Atributo de calidad observable en tiempo de ejecución. Se relaciona con throughput, latencia, utilización de recursos, etc.

- Métricas clave
- Throughput: peticiones/tiempo
- Latencia: tiempo de respuesta
- Uptime, Bandwidth, Errores, Garbage Collector
- Cómo medirlo
- Experimental: JMeter, Locust, K6, New Relic
- Teórico: teoría de colas, simulaciones, cadenas de Markov



Repasos adicionales

Performance



Tácticas de mejora:

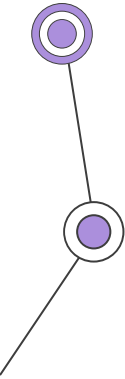
más máquinas, concurrencia, cachés
timeouts, rate limiting

1. **Caché y red:**

- a. Desafíos: sincronización, warm-up, políticas de expiración
- b. Estilos como Microservicios sacrifican rendimiento por flexibilidad

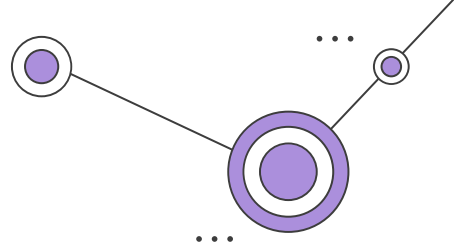
2. **Caso Amazon Prime:**

Volvieron parcialmente al monolito → Redujeron 90% del costo por latencia y red



Repasos adicionales

NO SQL



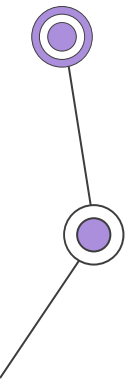
¿Por qué NoSQL?

- SQL tiene schema rígido, difícil de escalar
- NoSQL busca performance, escalabilidad y simplicidad
- BASE vs ACID

BASE = Eventually consistent

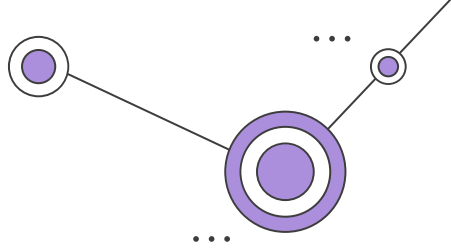
ACID = Transacciones fuertes

- Tipos de NoSQL:
- Key-Value: Redis, Dynamo
- Documental: MongoDB
- Columnar: BigTable, Cassandra
- Grafos: Neo4j
- Vectorial: búsquedas por embeddings



Repasos adicionales

NO SQL

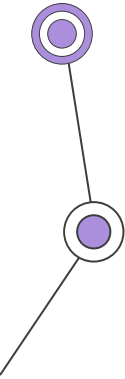


MongoDB vs SQL

- Tabla ↔ Colección, Fila ↔ Documento JSON, Joins ↔ Embedding
- Consideraciones tácticas y estratégicas
- ¿Qué tipo de dato tienes? ¿Read-heavy o write-heavy?
- ¿Tu equipo domina el stack? ¿Requieres alta consistencia?
- NewSQL

Intenta combinar escalabilidad de NoSQL con consistencia de SQL

Ej: CockroachDB, Aurora, Spanner



A decorative network diagram with purple nodes and lines. The nodes are represented by concentric circles, with some having a solid purple center and others being hollow. They are connected by thin black lines. The diagram is positioned around a central light purple blob containing the text.

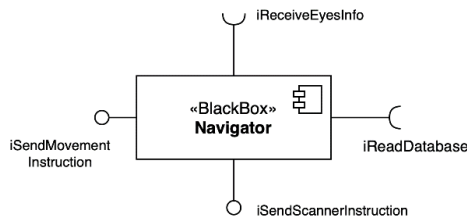
**Vamos al
caso**

Examen 2024-2

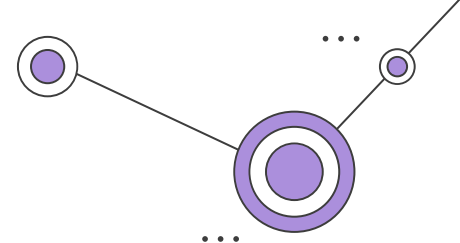
En el año 2032, un equipo de científicos e ingenieros de LASA, quieren crear un sistema para poder analizar y tomar muestras de las lunas de Júpiter (alrededor de 95), y así recabar datos de todas para poder verificar las condiciones reales que tienen para formar vida y ser los mejores para otras tareas como extracción de minerales. Así, su equipo les pide que hagan el diseño del sistema que analizará todo esto.

Para poder gestionar la información de todas estas lunas, se les avisa que existe un satélite que tiene conexión con distintos lugares de las lunas (pero no asegura conexión en la totalidad de la superficie de estas). Este será el encargado de recibir la información, y tendrá 8 computadores dentro que analizarán la información. Además, existirán personas dentro que deberán poder ver toda esta información a través de dashboards y tener un control humano por si algo falla.

En cada luna, se enviará una flota de robots con capacidad de navegar en distintos ambientes y tomar información de suelos, la atmósfera y elementos no identificados usando su propia base de datos llamados EVA (no los de Evangelion). Estos robots, se moverán autónomamente usando un componente Navigator en su sistema que se ve en la imagen. Además, para analizar los elementos de la luna los van a escanear y obtendrán sus métricas para guardarlas en su memoria interna conectada al componente Navigator, así no volverán a escanear elementos demasiado similares. Finalmente, estos robots se conectarán entre ellos para así transmitir la información de sus escaneos entre sí, formando una red. Si uno de los EVA se logra conectar con el satélite le avisará a su vecino que tiene la conexión para que sea el punto de conexión entre los EVA cercanos y el satélite.

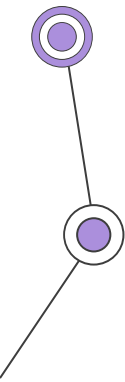


Examen 2024-2

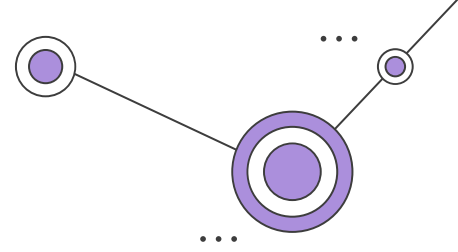


Dentro del satélite, como se mencionó antes, existen 8 computadores capaces de analizar la información. El satélite debe recibir la información y enviarla a cada uno de estos computadores en base a si les servirá el dato o no. Tendrán computadores que analizarán la cantidad de agua detectada, otros los tipos de rocas que se encuentran, otros los elementos químicos disponibles en la luna, etc, denominados equipos A hasta la H. Cada uno de estos computadores sabrá qué parte de la información que le debería llegar es la que deben usar, y tendrá que adaptarla para poder procesarla.

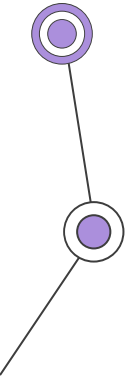
Finalmente, les informan que su sistema debe manejar muchos volúmenes de datos rápidamente, en donde deben considerar que una falla en la recopilación puede implicar que manden una colonia de personas a una zona inhabitable.



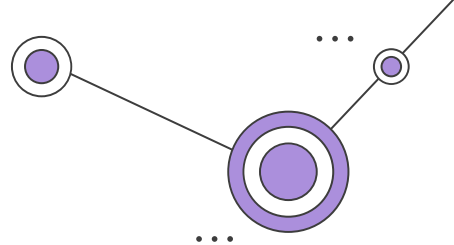
Pregunta 1



1)(3 ptos) Indique qué herramientas, estilos de arquitectura o tecnología específica sería la mejor para el sistema interno del satélite que propaga la información a cada equipo.

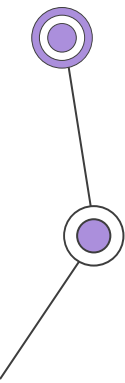


Pregunta 1

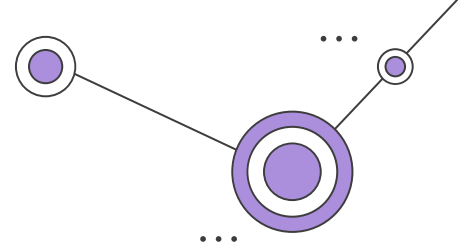


Event bus o broker

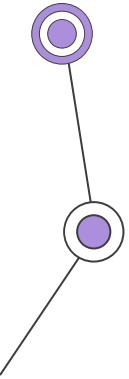
- Alto volumen de datos → puede recibir y distribuir millones de eventos sin bloquear.
- Desacoplamiento → los equipos A-H no dependen entre sí ni del origen de los datos.
- Persistencia y replay → si un equipo falla, puede reprocesar sin perder información crítica.
- Tolerancia a fallas → el sistema sigue funcionando aunque uno de los computadores caiga.
- Filtrado inteligente por tópicos → cada equipo recibe solo los datos que necesita.
- Escalabilidad interna → permite agregar más instancias de un equipo si aumenta la carga.
- Mejor que pub/sub simple porque:
 1. pub/sub no almacena mensajes
 2. pierde datos si nadie escucha
 3. no permite replay
 4. tiene menor confiabilidad para sistemas críticos.



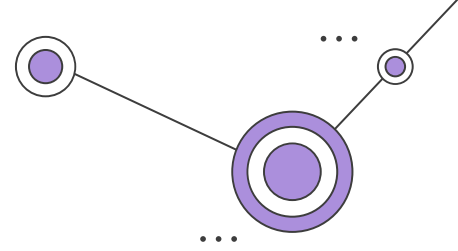
Pregunta 2



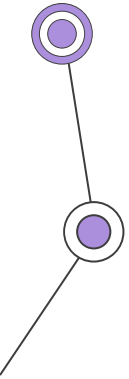
2)(3 ptos) Indique y justifique qué tipo de base de datos podría beneficiar de mejor manera el almacenamiento de las distintas métricas procesadas de cada equipo para luego ser leídas por las personas. Asuma que serán muchas para cada equipo y que deberán poder actualizar rápidamente los dashboard.



Pregunta 2



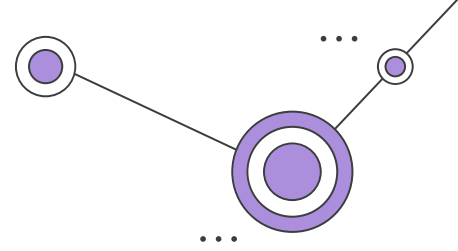
- Base de datos No SQL columnar. Esto debido a su capacidad de funcionar para guardar grandes cantidades de datos y poder usar sus capacidades para filtrar rápidamente por columnas en vez de por filas.
- **Caché por si solo es 1/3 puntos**, ya que eso no habla de cómo se guardan los datos, si no que de cómo el dashboard puede prevenir hacer tantas llamadas. Además, no soluciona el que si aplico un filtro en un dashboard por ejemplo, la request cambia por lo que no puedo usar una key para obtener datos antiguos, tengo que hacerlo a las bases de datos.



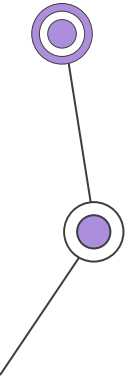
Pregunta 2

- Base de datos No SQL columnar. Esto debido a su capacidad de funcionar para guardar grandes cantidades de datos y poder usar sus capacidades para filtrar rápidamente por columnas en vez de por filas.
- **Caché por si solo es 1/3 puntos**, ya que eso no habla de cómo se guardan los datos, si no que de cómo el dashboard puede prevenir hacer tantas llamadas. Además, no soluciona el que si aplico un filtro en un dashboard por ejemplo, la request cambia por lo que no puedo usar una key para obtener datos antiguos, tengo que hacerlo a las bases de datos.

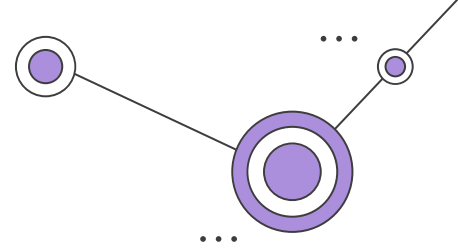
Pregunta 3



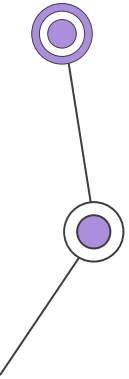
- Un miembro de su equipo les propone si ¿se pudiera usar un sistema interno de los EVA para que el envío de información no detenga las opciones de movimiento?, para los casos donde, luego de un análisis, deba escapar de un lugar sin esperar al envío, haciéndolo asíncrono. Indique su respuesta a esta pregunta y la justificación y medidas a tomar si se implementara o no para ese problema que indica. Su respuesta tendrá que estar reflejada en su diagrama.



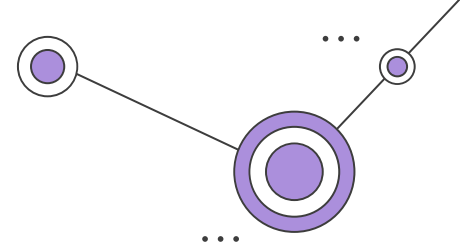
Pregunta 3



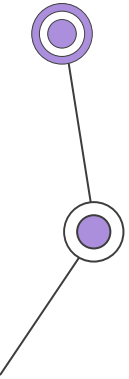
- Si se puede y se debería hacer al menos para que no sea síncrono en el momento, ya que un bloqueo en el envío puede generar que un robot se pierda o sea dañado si no se puede mover en ese momento.
- La implementación de esto se puede:
 - > Asincronía en base a eventos: Se debe indicar el cómo, la justificación, que puede hacer para implementarlo y asegurar su funcionamiento. Es válido indicar uso de workers para la propagación, o uso de Pub/Sub o EDA.
 - > Síncrono, pero con capacidad de esperar a que se encuentre seguro para enviarlo: Solución más complicada, pero puede ser que el EVA dependa de un componente de seguridad en base a su sistema de I/O de visión, donde si se detecta peligro se pueda abortar el sistema de envío para dar prioridad a la navegación. En este caso se debe tener en cuenta que la información que no se envió se debe guardar para luego ser propagada correctamente, y que aparte de poder bloquearse y detenerse, debe poder reanudarse.



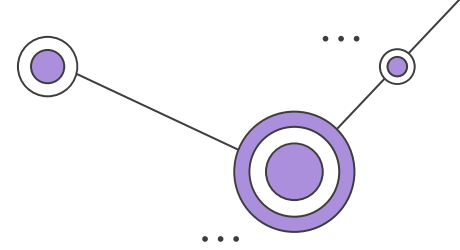
Pregunta 4



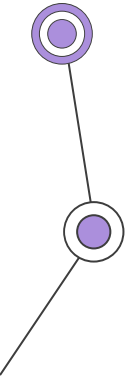
- (2 ptos) Indique las características arquitectónicas que Ud. considere son fundamentales en este problema. Enumérelas en orden de prioridad y justifique brevemente.



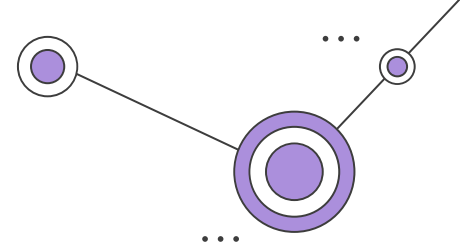
Pregunta 4



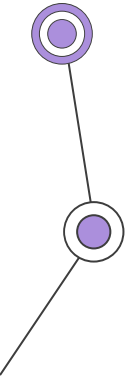
- (2 ptos) Indique las características arquitectónicas que Ud. considere son fundamentales en este problema. Enumérelas en orden de prioridad y justifique brevemente.
- Escalabilidad
- Performance
- Integrabilidad
- Confiabilidad
- Portabilidad – Solo si lo mencionan por ejecutar el código en distintos ambientes por los computadores. Si es solo por los EVA está malo, portabilidad no es lo mismo que el código se ejecute en un ambiente que se moverá espacialmente.



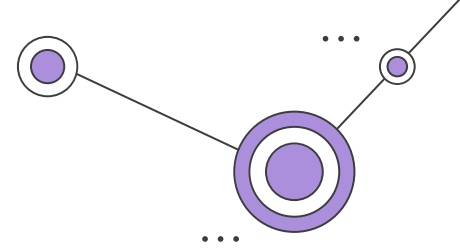
Pregunta 5



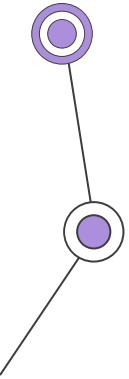
- Indique los estilos/patrones arquitectónicos que va a emplear para solucionar el problema. Indique explícitamente qué aspecto del problema ataca cada estilo (responsabilidad).



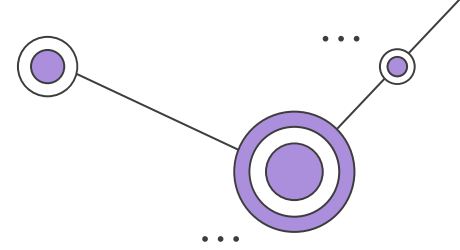
Pregunta 5



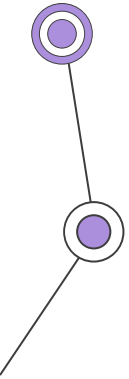
- Indique los estilos/patrones arquitectónicos que va a emplear para solucionar el problema. Indique explícitamente qué aspecto del problema ataca cada estilo (responsabilidad).
- Peer To Peer
- Basado en eventos – Event Bus
- Cliente servidor – Solo si lo justifican como satélite con EVAs
- En lotes secuenciales – Procesamiento de datos en equipos A-H
- Otro bien justificado

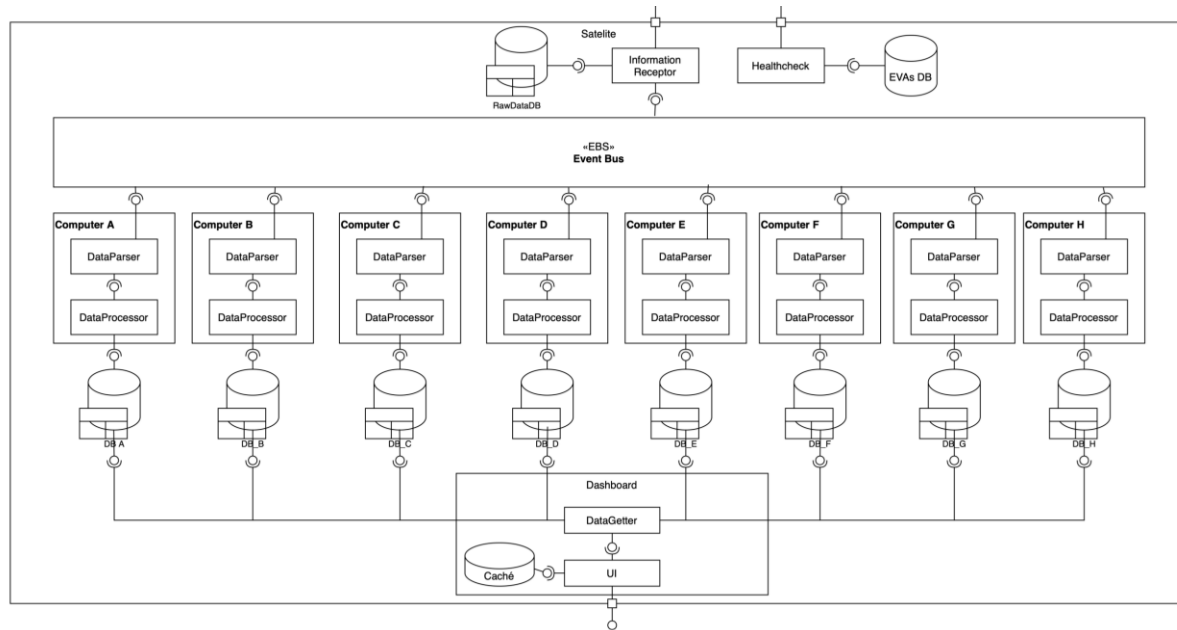


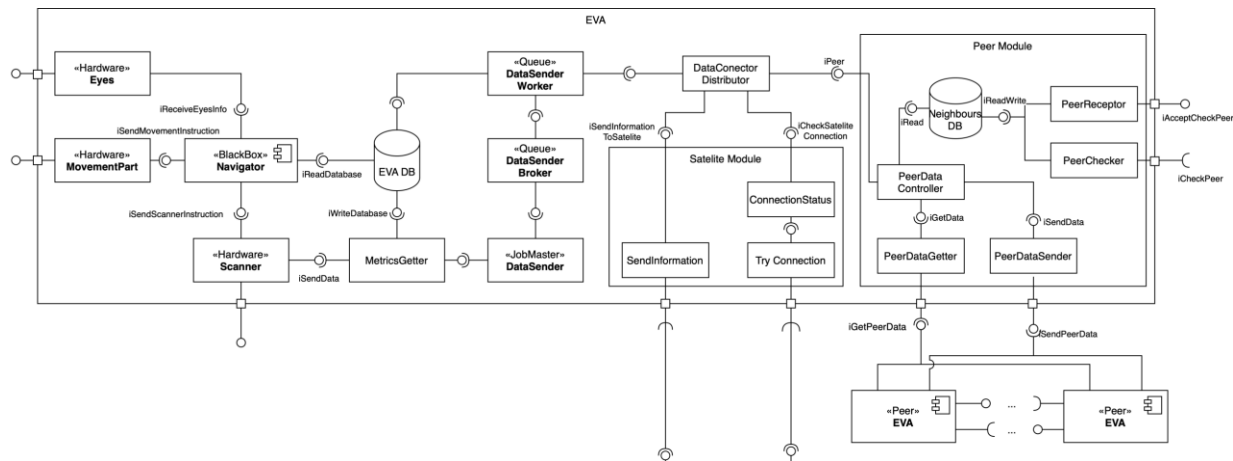
Pregunta 6

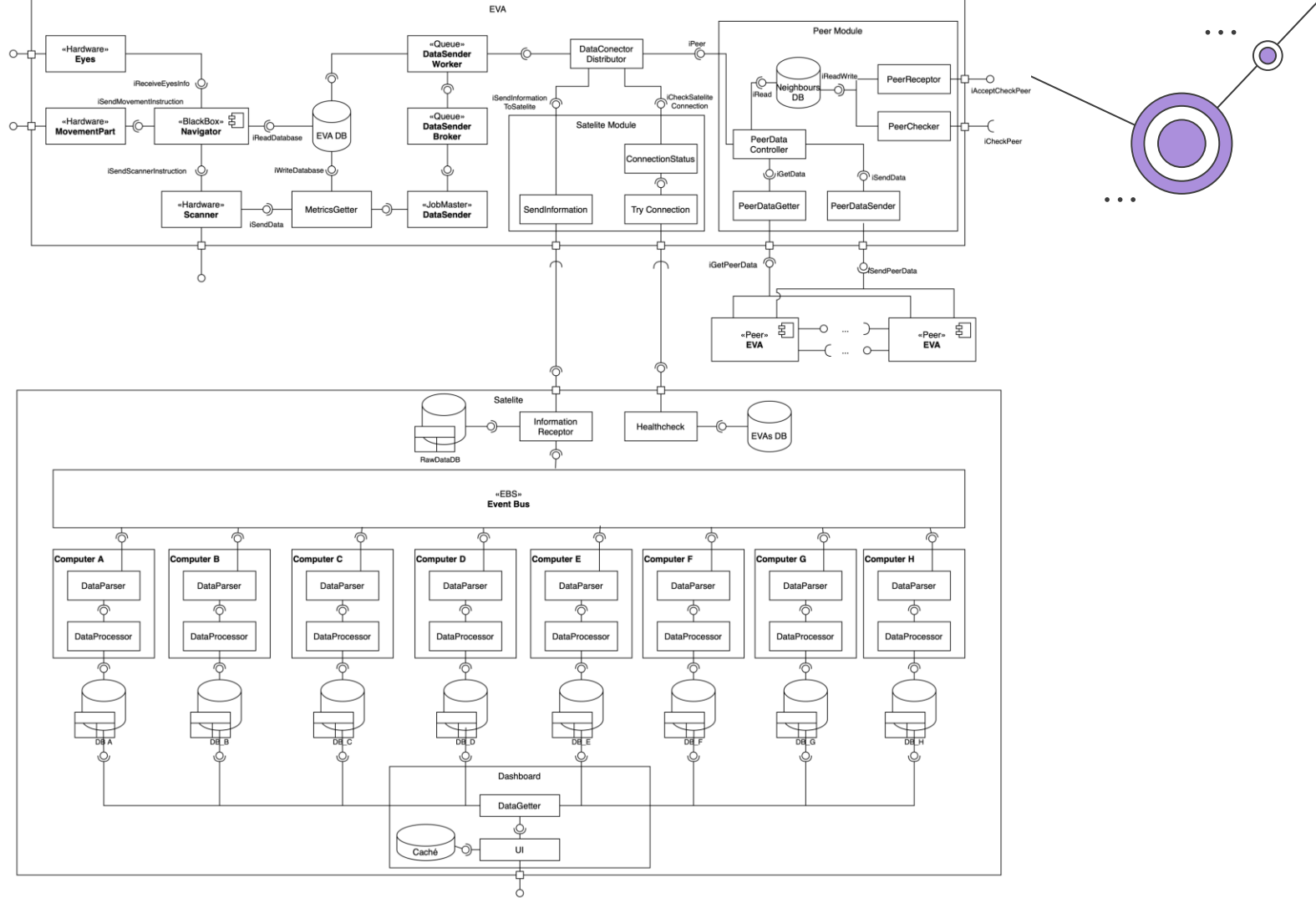


- Confeccione un diagrama de componentes UML indicando **los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente**, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Diseñelo en una hoja aparte.



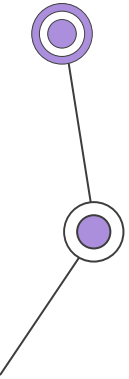
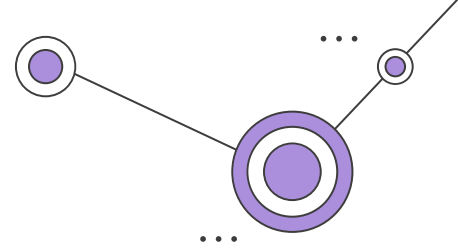


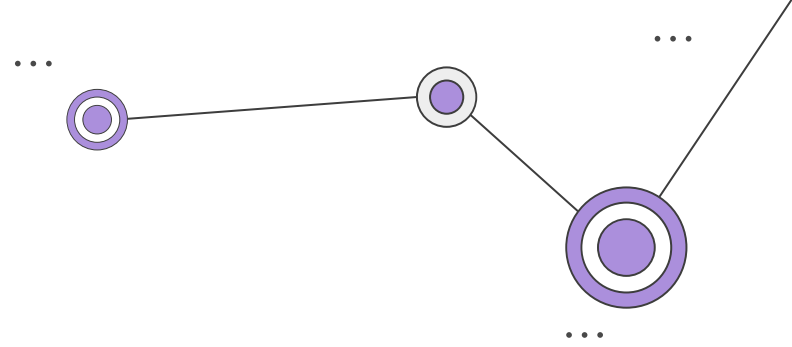
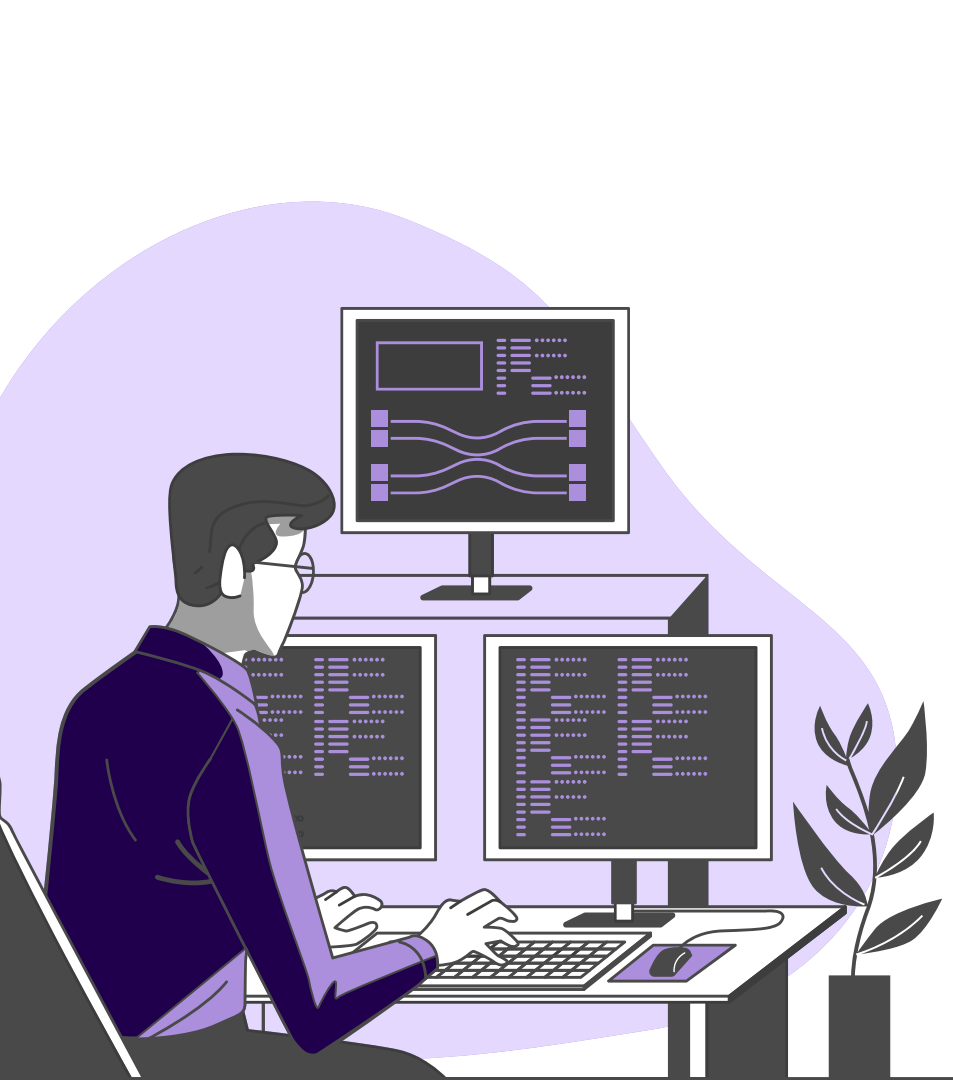




Importante

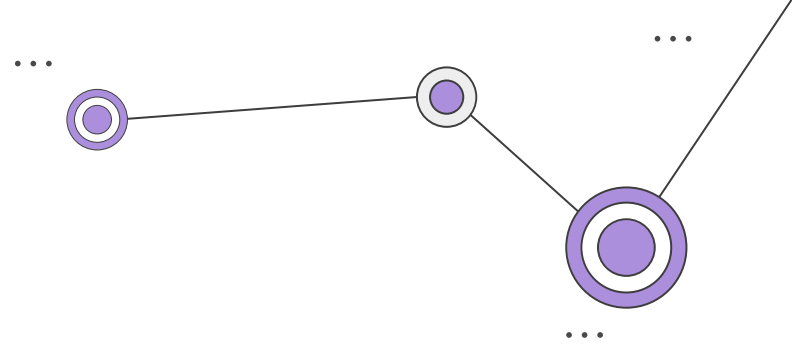
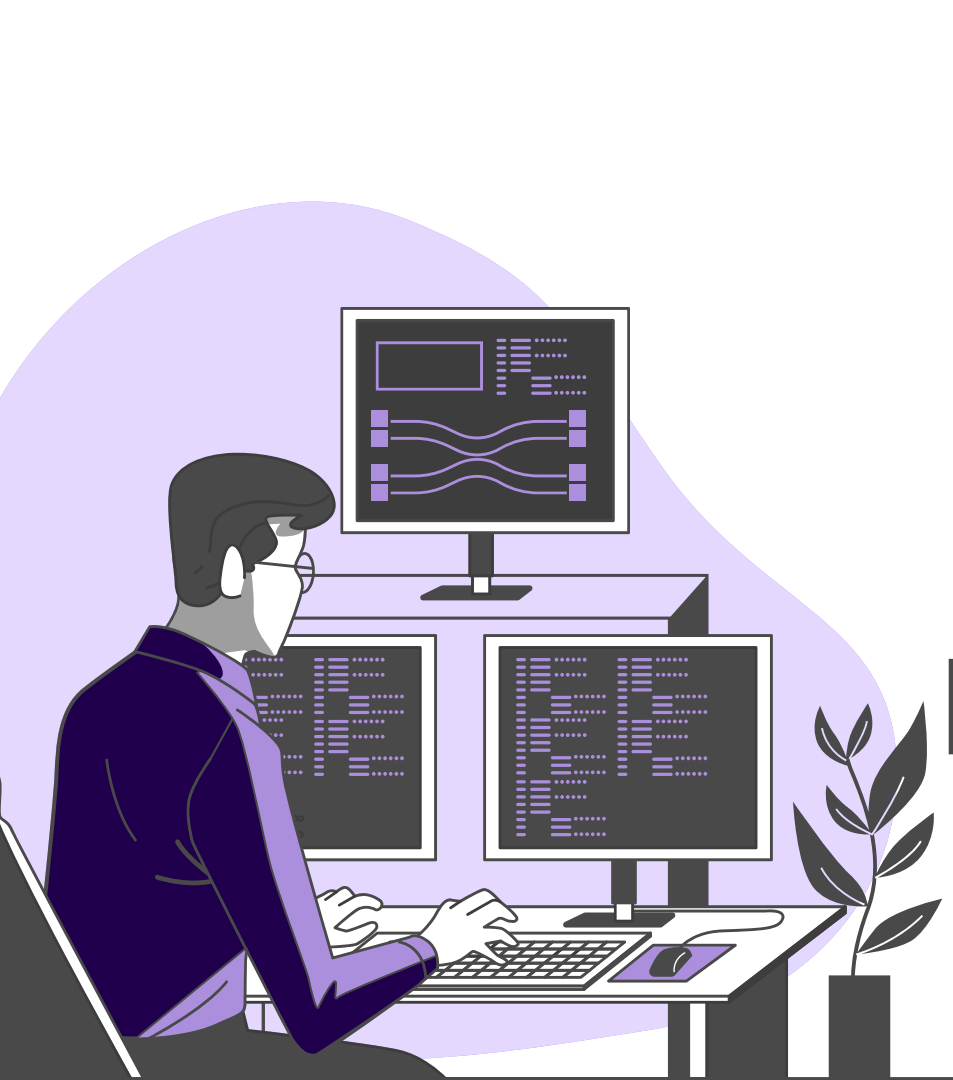
- Ser consistentes con sus NFRS y Estilos definidos previamente
- Tener componentes conectados
- Seguir estilo de UML visto en clases / ayudantías





Recomendaciones:

1. Lean con calma el enunciado
2. Repasen ayudantías/ Interrogaciones pasadas
3. Apliquen lo que han aprendido en clases/ ayudantías, no busquen online sus respuestas



Ayudantía 12: Diagramación UML 5

Martín Eyzaguirre

